

Plan de verificación: “Dark RISC-V”

Autores:

Sebastian Herrera Esquivel

Dennis Chavaría Soto

Ricardo Mena Cortés.

Jesús Jiménez Ulate

15 de Julio del 2024

Revisiones

Número de Revisión	Fecha de revisión	Descripción	Autor
#1	Julio 5	Creación de estrategia de verificación, niveles y descripción del ambiente.	Jesús.
#2	Julio 8	Descripción de la metodología.	Ricardo.
# 3	Julio 9	Ambiente de Verificación	Dennis.
#4	Julio 10	Alcance	Sebastian
#5	Julio 15	Revision Final del Documento	Sebastian, Dennis, Ricardo y Jesús.

Table of Contents

Estrategia de Verificación	4
Niveles de Verificación	4
Ambiente de Verificación	6
Alcance del Plan de Verificación	8
Requerimientos de Cobertura y Métricas	9
Función Reset	11
Función Clock	12
Función 1 R	12
Función 2 I	13
Función 3 L	14
Función 4 S	15
Función 5 B	16
Función 6 J	16
Función 7 U	17
Función 8 C	FALTA!!!

1. Estrategia de verificación:

a. Describir si es aleatoria o dirigida.

Se realizará una estrategia de verificación pseudo-aleatoria que permita generar secuencias de instrucciones que no lleguen a crear una inconsistencia en los datos leídos desde la memoria.

b. Describir si es caja negra, blanca o gris.

La estrategia de verificación aplicada se caracteriza por ser aleatoria, con capacidad de pruebas semi-dirigidas que limitan los rangos de aleatorización a tipos de instrucciones específicas del ISA RISC-V.

De acuerdo con lo visto en clase, la estrategia de verificación es de caja blanca porque se tiene total acceso a señales internas del system-on-chip, además, se tiene conocimiento sobre cómo funcionan y en qué momento realizan determinadas acciones.

2. Niveles de verificación

a. Descripción de la jerarquía de verificación

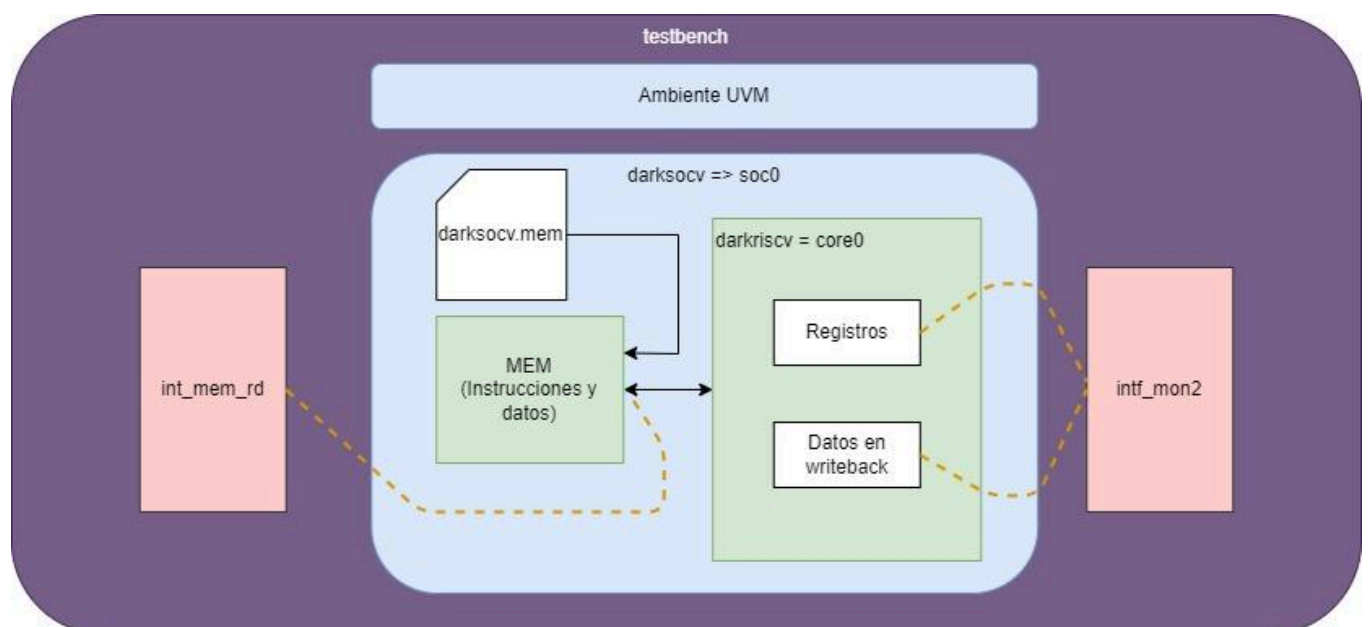


Figura 1: Jerarquía de verificación.

La figura 1 ilustra la implementación del Dispositivo bajo Verificación, darkriscv, dentro del módulo de mayor jerarquía, darksocv, así como los componentes

relevantes en el presente proyecto. Alrededor se encuentran algunos bloques de verificación del ambiente propuesto. El RTL se compone de un bloque principal System-On-Chip (soc0) que instancia los siguientes módulos:

1. **DarkRiscv:** Dispositivo bajo verificación. Se trata del procesador que implementa los diferentes tipos de instrucciones del ISA.
2. **Arreglo de memoria:** Bloque que almacena las instrucciones para procesar y funciona como memoria de datos.
3. **Unidad UART:** Fuera del alcance del proyecto. Implementa una interfaz de comunicación de acuerdo con el protocolo UART.
4. **Darkpll:** Fuera del alcance del proyecto. Implementa un sistema de generación de clocks orientado a ser usado en placas de desarrollo o FPGAs.

El nivel de verificación es de subsistema, debido a que, dentro del SOC se encuentra el Dispositivo Bajo Verificación(DUV) que interactúa con la memoria. Ambos componentes son implementados mediante código sintetizable RTL, de este modo, la memoria no está integrada mediante un BFM.

El enfoque del proyecto recae sobre el módulo darkriscv, porque es el nivel jerárquico más bajo que contiene las funciones por verificar, es decir, las instrucciones del ISA. La interacción a dicho nivel permite tener completa controlabilidad sobre las instrucciones que son cargadas al DUV.

3. Ambiente de verificación:

a. Describir nivel de abstracción, capas y metodología.

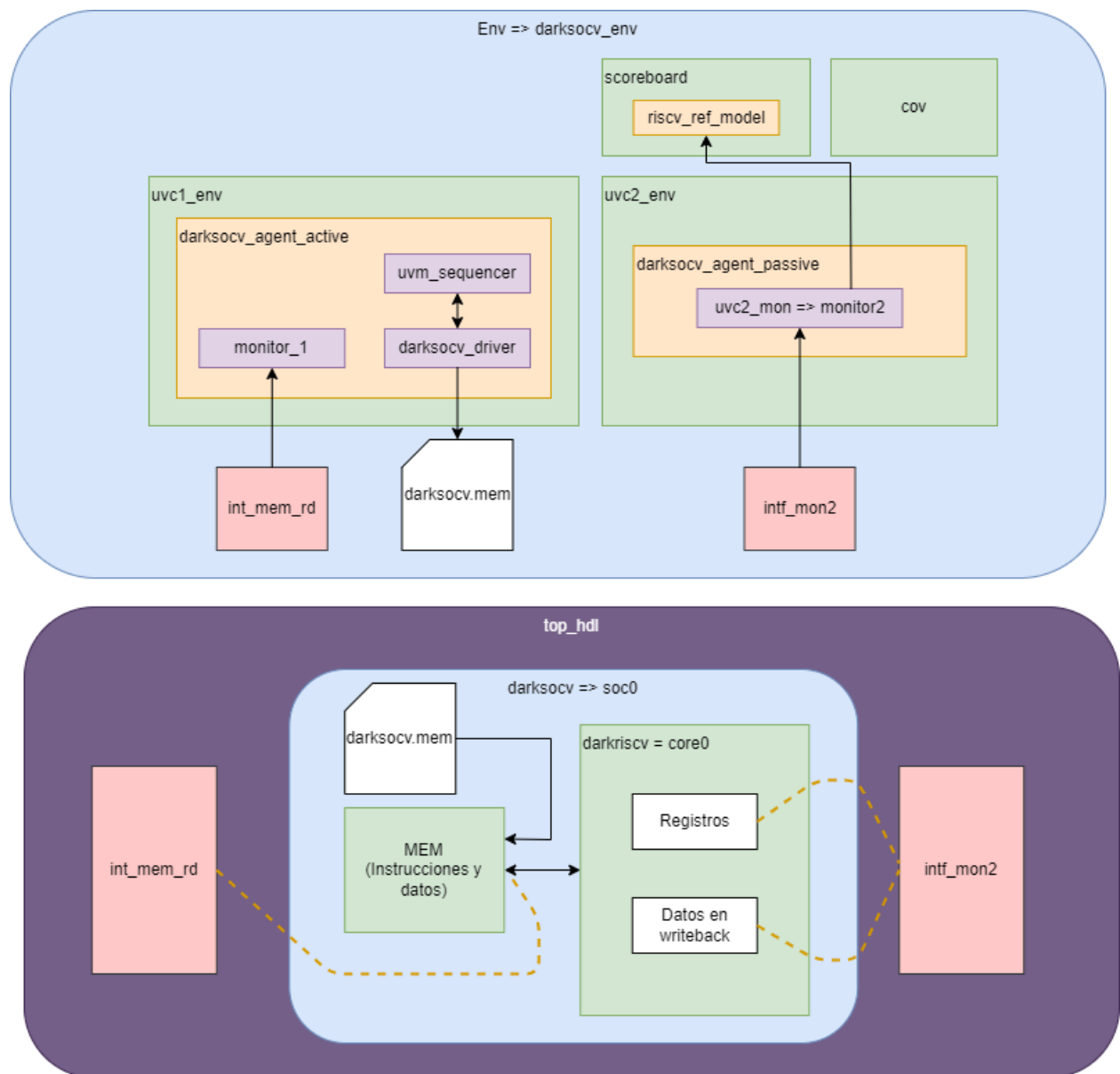


Figura 2: Ambiente de verificación implementado.

Respecto al nivel de abstracción, los estímulos se controlan a través de un programa generado por una secuencia que crea las instrucciones en formato hexadecimal de acuerdo con los campos y parámetros que definen una instrucción del ISA. Dicho programa se almacena en el archivo de texto `darksocv.mem`, ilustrado por la Figura 2.

El testbench, a su vez, tiene niveles de abstracción respecto a la comunicación de los bloques del ambiente mediante los siguientes TLMs:

- **TLM Monitor 1-Scoreboard:** Utilizado para la transferencia del monitor 1 al scoreboard de una instrucción decodificada que deberá ser procesada.
- **TLM Monitor 2-Scoreboard:** Utilizado para el envío al scoreboard de los datos que resultan del procesamiento de una instrucción por el DUT.

Por otra parte, el testbench cuenta con un total de tres interfaces:

- **Interfaz de instrucciones leídas:** Interfaz virtual implementada para el monitoreo de la instrucción que procesará el “core” y el “program counter” asociado.
- **Interfaz de monitor 2:** Interfaz virtual para el monitoreo de la instrucción que procesará el “core” y el “program counter” asociado.
- **Interfaz del SOC:** Interfaz virtual utilizada por los componentes de verificación para acceder a la señal del reloj y tener una referencia de tiempo. También permite al driver controlar el reset y dar valores bien conocidos a la interfaz UART que no se utiliza.

Sobre las capas del ambiente de verificación, se utiliza un esquema “*doble-top*”; en este, el RTL y las interfaces virtuales se instancian dentro de un “*top_hdl*”, por su parte, el ambiente de verificación se integra en un módulo “*top_hvl*”. Este instancia la primer capa, el ambiente (darksocv_env) con las siguientes capas:

- **Scoreboard:** Esta capa contiene la lógica para operar los TLM que reciben las instrucciones que deberían ser procesadas, así como los resultados de las instrucciones ya procesadas.
 - *Riscv_ref_model*: Subcapa del scoreboard que implementa el comportamiento teórico según las instrucciones del ISA RISC-V.
- **UVC1_ENV:** Contiene la capa de agente activo. Esta última, a su vez, contiene la capa de monitoreo de instrucciones por procesar (monitor 1), la capa del *sequencer* (uvm_sequencer) y el *driver* (darksocv_driver) que almacena las instrucciones generadas en un archivo de texto que es procesado por el core.
- **UVC2_ENV:** Esta contiene la capa de agente pasivo que, a su vez, instancia la capa de monitoreo de instrucciones procesadas (monitor2).

Metodología

Para las instrucciones R, I, load, store, J B, U se seguirá la siguiente estrategia:

1. Utilizar una secuencia que genere únicamente el tipo de instrucción deseado.
2. Completar la secuencia de reset y permitir que el procesador ejecute el programa.
3. Utilizar el scoreboard-checker para verificar que el procesador cumple con los resultados esperados de acuerdo con la especificación.
4. Utilizar los puntos de cobertura específicos para cada función de verificación para identificar el espacio de estados de las instrucciones ejercitadas.
5. Analizar los resultados obtenidos y reajustar en caso de ser necesario.

Para la verificación de las funciones mixtas, la secuencia no deberá estar limitada a aleatorización un solo tipo de instrucción, en cambio, podrá generar todas las opciones.

Metodología para cobertura estructural

Ejecutar múltiples veces un test básico que prueba todas las funciones del sistema y recolectar la cobertura estructural de interés. Es necesario que cada simulación varíe únicamente en la semilla de aleatorización.

4. Alcance del plan de verificación

Se utilizará la arquitectura de RISC-V RV32I la cual es una arquitectura de conjunto de instrucciones (ISA) de código abierto que define un núcleo de instrucciones base de 32 bits, diseñada para ser modular, escalable y fácil de implementar.

Para lograr una cobertura completa del procesador DARK RISC-V. La verificación abarca:

- **Funcionalidad de instrucciones:** Se valida el correcto funcionamiento de diversos tipos de instrucciones, incluyendo
 - Registro a registro (R).
 - Inmediatas (I).
 - De salvado de datos (I-L).
 - De carga de datos (S).
 - De salto condicional (B).
 - Carga y agregado de la parte alta de un valor inmediato (U).
 - De salto incondicional(J).

- **Manejo de registros:** Se comprueba la actualización y uso adecuados de los registros del procesador.
- **Operaciones de memoria:** Se verifica la gestión precisa de valores inmediatos y operaciones de memoria.
- **Transiciones entre instrucciones:** Se valida la transición correcta entre instrucciones, asegurando un flujo de ejecución temporal coherente.
- **Verificación de Señales de Control y Funcionalidad FLUSH:** Se verifica el correcto funcionamiento de las señales de control, como BE, FLUSH, JREQ, y las banderas de instrucciones, así como la funcionalidad de la operación FLUSH.

La exhaustiva verificación realizada mediante este proyecto asegura el estado de la integridad y robustez del procesador DARK RISC-V ante variaciones operativas y diversos programas, garantizando un funcionamiento confiable y predecible.

5. Requerimientos de cobertura y métricas.

a. Describir tipo de cobertura estructural y funcional a utilizar.

Cobertura Estructural:

- **Cobertura de línea:** Se utilizará la herramienta Covered y DSIM para medir la cobertura de línea del DUV. El objetivo es alcanzar una cobertura de línea de al menos el 82% en las partes de código relevantes para el proyecto. Esto depende de los macros ``define` y los parámetros de los bloques *generate* utilizados.
- **Cobertura de toggle:** Se medirá la cobertura de las señales críticas del procesador DARK RISC-V, incluyendo señales de control, datos, reloj y reinicio. El objetivo es alcanzar una cobertura de *toggle* del 82% para todas las señales relevantes.
- **Cobertura de transición de estados(instrucciones):** Se medirá la cobertura de la transición de estados del procesador DARK RISC-V (transición de instrucciones) por medio de la herramienta covered y DSIM. El objetivo es alcanzar una cobertura de transiciones de estado

(instrucciones) del 82% para todas las transiciones de estado posibles válidas.

Cobertura Funcional:

- **Cobertura de instrucciones:** Se utilizarán “*cover points*” y aserciones para medir la cobertura de las instrucciones del procesador DARK RISC-V. El objetivo es alcanzar una cobertura de instrucciones del 82% para todos los tipos de instrucciones (R, I, I-L, S, B, J, U).
- **Cobertura de memoria:** Se utilizarán “*cover points*” para medir la cobertura del uso de memoria y banco de registros del procesador DARK RISC-V, con el objetivo de alcanzar una cobertura del 82% para todas las posiciones de memoria y registros posibles. Esta evaluación es crucial, especialmente para las posiciones fronterizas e intermedias.

b. Definir metas del proyecto.

- **Garantizar la correcta funcionalidad del procesador DARK RISC-V:** La verificación exhaustiva debe asegurar que el procesador funcione de acuerdo con las especificaciones de diseño y se comporte de manera predecible en todas las condiciones posibles.
- **Identificar y corregir errores de diseño:** La verificación debe detectar y ayudar a corregir errores de diseño en el procesador DARK RISC-V.
- **Mejorar la calidad y confiabilidad del sistema:** Una verificación completa aumenta la confianza en la calidad y confiabilidad del procesador DARK RISC-V, lo que reduce el riesgo de fallos en el producto final. En este caso, se verifica el correcto funcionamiento de características esenciales, incluyendo las instrucciones R, I, J, B, L, S y U.

c. Métricas y Análisis

- **Cobertura Estructural mediante la herramienta covered**
 - **CL para generar archivos de cobertura:**
 - `covered score -t top -v rtl/config.vh -v rtl/top_hdl.sv -v rtl/darksocv.v -v rtl/darkriscv.v -e darkpll -e darkuart -vcd sim/darksocv.vcd`
 - **CL para generar archivos de reporte:**
 - `covered report -view cov.cdd`
- **Cobertura Funcional y Estructural mediante la herramienta DSIM**
 - Se genera un script que tome la base de datos de cobertura generada por DSIM y la convierta un archivo fuente html para desplegar posteriormente la información
 - Se construye el reporte con el comando “dcreport” y se abre el reporte para su consulta
 - `dcreport -out_dir "$OUT_DIR" "$METRICS_DB"`
 - `open "$INDEX_HTML"`
 - Se hace merge de los archivos de cobertura con el comando **dcmerge**.
- Cobertura de Transiciones Funcional, se verifica que ocurren todos los tipos de transiciones de tipos de funciones, corriendo la semilla testbasic.

covergroup cov_transition[no per-instance data saved]

covergroup	100.00
cvr_current_instr	100.00

covergroup cov_B[no per-instance data saved]

covergroup	27.58
cvr_instr	75.00
cvr_rs1	62.50
cvr_rs2	56.25
cvr_rs1_value_un	28.57
cvr_rs1_value_sig	14.29
cvr_rs2_value_un	28.57
cvr_rs2_value_sig	14.29
cvr_imm_un	14.29
cvr_imm_sig	14.29
cvr_rd	25.00
cvr_pc	100.00
cvr_x_ints_rs1	9.77
cvr_x_ints_rs2	9.77
cvr_x_ints_rd	8.20
cross_imm_sig_pc	4.33
cross_instr_rs1_un_rs2_un	2.03
cross_instr_rs1_sig_rs2_sig	1.69

covergroup cov_J[no per-instance data saved]

covergroup	33.98
cvr_instr	50.00
cvr_rs1	6.25
cvr_rs1_value_un	14.29
cvr_imm_sig	14.29
cvr_rd	100.00
cvr_pc	100.00
cvr_x_ints_rs1	3.12
cvr_x_ints_rd	50.00
cross_imm_sig_pc	0.76
cross_imm_rs1_un	1.09

covergroup cov_U[no per-instance data saved]

covergroup	99.36
cvr_instr	100.00
cvr_imm_sig	100.00
cvr_rd	100.00
cvr_pc	100.00
cvr_x_ints_rd	100.00
cross_imm_sig_pc	96.18

covergroup cov_Load[no per-instance data saved]

covergroup	73.99
cvr_instr	83.33
cvr_rs1	96.88
cvr_rs1_value_un	14.29
cvr_imm_sig	71.43
cvr_datai	100.00
cvr_ldata	100.00
cvr_rd	96.88
cvr_x_ints_rs1	54.17
cvr_x_ints_rd	55.21
cross_instr_datai	41.67
cross_imm_sig_rs1_un	100.00

covergroup cov_S[no per-instance data saved]

covergroup	83.95
cvr_instr	100.00
cvr_rs1	96.88
cvr_rs2	100.00
cvr_rs1_value_un	14.29
cvr_rs2_value_un	71.43
cvr_imm_sig	71.43
cvr_rd	93.75
cvr_sdata	100.00
cvrx_ints_rs1	78.12
cvrx_ints_rs2	82.29
cvrx_ints_rd	51.04
cross_instr_imm_sig	100.00
cross_instr_rs1_un	100.00
cross_instr_rs2_un	100.00
cross_imm_sig_rs1_un	100.00

Functional coverage for \$unit.funct_coverage

covergroup cov_R[no per-instance data saved]

covergroup	87.81
cvt_instr	100.00
cvt_rs1	100.00
cvt_rs2	100.00
cvt_rd	100.00
cvtx_ints_rs1	70.94
cvtx_ints_rs2	70.62
cvtx_ints_rd	73.12

covergroup cov_R_SLL[no per-instance data saved]

covergroup	67.19
cvt_rs1_sll_values	100.00
cvt_rs2_sll_shift	34.38

covergroup cov_R_SRA[no per-instance data saved]

covergroup	54.17
cvt_rs1_sra_values	83.33
cvt_rs2_sra_shift	25.00

covergroup cov_R_SRL[no per-instance data saved]

covergroup	45.83
cvt_rs1_srl_values	66.67
cvt_rs2_srl_shift	25.00

covergroup cov_R_SLT[no per-instance data saved]

covergroup	89.58
cvt_rs1_slt_values	100.00
cvt_rs2_slt_values	100.00
cvtx_slt_rs1_rs2	68.75

covergroup cov_R_SLTU[no per-instance data saved]

covergroup	91.67
cvt_rs1_sltu_values	100.00
cvt_rs2_sltu_values	100.00
cvtx_sltu_rs1_rs2	75.00

[illegible]

```

::
:: Covered -- Verilog Coverage Summarized Report
::
::
:::

```

GENERAL INFORMATION

* Report generated from CDD file : /home/aqua/Desktop/cover_line/cov_branch1.cdd

* Reported by : Module

* Report generated from CDD file that was merged from the following files:

Filename	Leading Hierarchy
cov_testbasic5.cdd	*
cov_testbasic5.cdd	*
cov_testbasic5.cdd	*
cov_testbasic5.cdd	*
cov_testbasic5.cdd	*

LINE COVERAGE RESULTS

Module/Task/Function	Filename	Hit/	Miss/Total	Percent hit
\$root	NA	0/	0/ 0	100%
top	top_hdl.sv	1/	0/ 1	100%
darksocv	darksocv.v	45/	20/ 65	69%
darkriscv	darkriscv.v	69/	0/ 69	100%
Accumulated		115/	20/ 135	85%

6. Reset:

a. Inicialización ("cold boot")

Función a verificar:	Inicialización
Sección de la especificación:	no aplica
Descripción:	Se realiza el reinicio (RES) durante el procesamiento de instrucciones.
Prioridad:	Crítica
Autor (Dueño de la función):	Dennis Chavarría
Requerimientos del banco de pruebas:	Se requiere la comunicación driver-sequencer con el espacio de memoria. La secuencia se limita a generar únicamente instrucciones de un solo tipo. Se ejecuta la inicialización de Reset y generación de clock. Pueden habilitarse puntos de cobertura. Se utiliza el scoreboard que tiene la capacidad de simular el flush de instrucciones y reinicio de valores en 0 en condiciones de "reset".
Pruebas:	Test general que inyecte instrucciones y dispare el reinicio durante la ejecución de un programa.
Criterio de chequeo:	Se cambia de estado de 1->0 y de 0->1, las instrucciones se ejecutan forma correcta y no se generan errores

Cobertura funcional:	No aplica.
Observaciones:	Este test se revisó de forma visual debido a las características del ambiente de verificación y ajuste de prioridades.

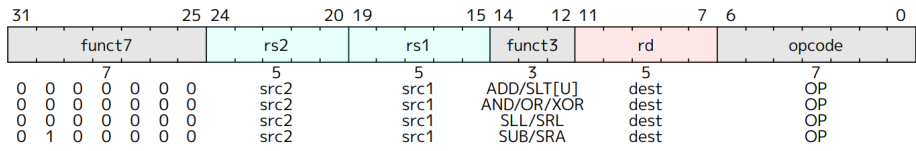
7. Clock:

a. Inicialización (“cold boot”).

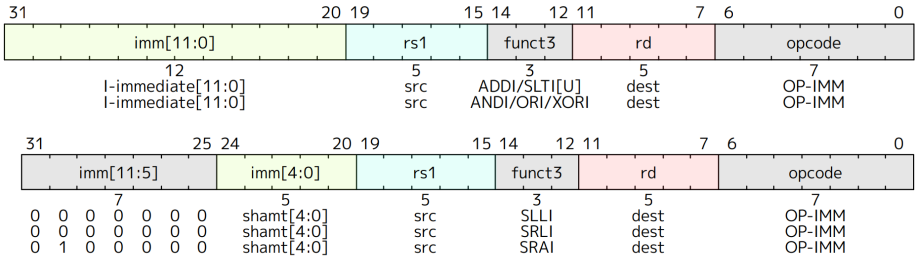
Función a verificar:	Inicialización
Sección de la especificación:	no aplica
Descripción:	CLK puede cambiar la frecuencia a algunos valores y ver si entran y salen las instrucciones esperadas.
Prioridad:	Crítica
Autor (Dueño de la función):	Ricardo Mena
Requerimientos del banco de pruebas:	Se requiere la comunicación driver-sequencer con el espacio de memoria. La secuencia genera las instrucciones para determinar si el procesador las ejecuta independientemente de la frecuencia. Se requiere cambiar la frecuencia del reloj de forma semialeatoria. Por limitaciones del compilador se utiliza un argumento de compilación pero debe agregarse manualmente.
Pruebas:	Se cambia la frecuencia de reloj por valores aleatorios en la prueba mixta.
Criterio de chequeo:	La frecuencia del clock varía, pero los checkers deben mostrar el correcto funcionamiento del RISCV aún cambiando la frecuencia.
Cobertura funcional:	Aserciones para verificar que no se incumple alguna condición crítica.
Observaciones:	Por limitaciones del compilador se utiliza un argumento de compilación pero debe agregarse manualmente.

8. Función #1

Función a verificar:	Instrucciones tipo R
Sección de la especificación:	Capítulo 2.4.2

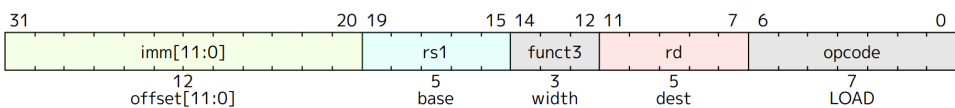
Descripción:	 Todas las operaciones leen los registros rs1 y rs2 como operandos fuente y escriben el resultado en el registro rd. Los campos funct7 y funct3 seleccionan el tipo de operación
Prioridad:	Crítica
Autor (Dueño de la función):	Ricardo Mena C
Requerimientos del banco de pruebas:	Se requiere la comunicación driver-sequencer con el espacio de memoria. La secuencia se limita a generar únicamente instrucciones tipo R. Inicialización de Reset y generación de clock, habilitar puntos de cobertura únicamente para instrucciones R. Se requiere de 31 instrucciones tipo I para la inicialización del banco de registros.
Pruebas:	Prueba aleatoria Dirigida: Generar un número determinado de instrucciones tipo R (func3/func7/OPCODE) y las direcciones de los registros rs1 , rs2 y rd a utilizar para la operación.
Criterio de chequeo:	El resultado de la operación entre rs1 y rs2 guardado en rd se compara con el resultado de la predicción realizada por el modelo de referencia.
Cobertura funcional:	Realizar operaciones con “<i>cover points</i>” que evalúen los siguientes parámetros: 1. ADD, AND, OR, SLL, SLT, SLTU, SRA, SRL, SUB, XOR 2. Registros fuente 3. Registros destino. 4. Hacer producto cruz de elemento 1x2 5. Hacer producto cruz de elemento 1x3
Observaciones:	Los casos de las variables DPTR, S1PRT, S2PTR se atienden con los “ <i>cover points</i> ” 2 y 3 que analizan los registros fuente y destino.

9. Función #2

Función a verificar:	Instrucciones tipo I
Sección de la especificación:	Capítulo 2.4.1
Descripción:	Formato de instrucción tipo I: 

	Se utiliza para codificar instrucciones con un operando inmediato de 12 bits con signo, con un rango de [-2048..2047], un registro destino (rd) y un registro fuente 1 (rs1). Se incluyen operaciones como corrimientos lógicos, aritméticos, suma y operaciones lógicas.
Prioridad:	Crítica
Autor (Dueño de la función):	Sebastian Herrera Esquivel
Requerimientos del banco de pruebas:	Comunicación <i>driver-sequencer</i> con el espacio de memoria. La secuencia debe estar limitada a generar únicamente instrucciones tipo I. Al iniciar el proceso de simulación, se realiza un reset y se genera el clock. Los puntos de cobertura solo se habilitan para instrucciones I.
Pruebas:	Prueba aleatoria Dirigida: Genera un número determinado de instrucciones tipo I (func3/OPCODE), valor Imm y las direcciones de los registros rs1 y rd a utilizar para la operación.
Criterio de chequeo:	El resultado de la operación entre rs1 y Imm guardado en rd se compara con el resultado de la predicción realizada por el modelo de referencia.
Cobertura funcional:	Realizar operaciones con “ <i>cover points</i> ” que evalúen los siguientes parámetros: 1. ADDI, ANDI, ORI, XORI SLLI, SLTI, SLTIU, SRAI, SRLI. 2. Registros fuente. 3. Registros destino. 4. Hacer producto cruz de 1x2. 5. Hacer producto cruz de 1x3.
Observaciones:	Los casos de las variables DPTR, S1PRT, S2PTR se atienden con los “ <i>cover points</i> ” 2 y 3.

10. Función #3

Función a verificar:	Instrucciones tipo LOAD
Sección de la especificación:	2.6
Descripción:	Formato de la instrucción:  Al ejecutar esta instrucción se carga en el registro <i>rd</i> el valor ubicado en la dirección calculada con el valor en el registro <i>rs1</i> más el offset con signo en el <i>imm</i>. Con el funct3 se selecciona si el dato leer es un <i>word</i> (32 bits), <i>half</i> (16 bits con o sin signo) o <i>byte</i> (8 bits con o sin signo)
Prioridad:	Crítica
Autor (Dueño de la función):	Dennis Chavarría
Requerimientos del banco de pruebas:	Se debe restringir la aleatorización para que la suma del valor en <i>rs1</i> y el <i>offset</i> , que da como resultado la dirección efectiva, esté dentro del rango de la memoria de datos, el cual es [2048, 4096]

Pruebas:	Se coloca previo a la instrucción <i>load</i> una instrucción <i>addi</i> y una <i>slli</i> para colocar un valor dentro del rango en el registro <i>base</i> , también se verifica que no se salga del rango al sumarse con el <i>offset</i>
Criterio de chequeo:	El valor cargado en el registro destino debe coincidir con el valor en la dirección efectiva de la memoria, el cual debe ser el mismo calculado por el modelo de referencia.
Cobertura funcional:	Verificar todas las instrucciones de carga (LB, LH, LW, LBU, LHU), los valores de los registros fuente y destino, y los valores inmediatos con signo. Además, cubrir todos los posibles valores de datos de entrada y de datos cargados, y asegurar la cobertura de direcciones de memoria entre 512 y 1023.
Observaciones :	

11. Función #4

Función a verificar:	Instrucciones tipo STORE
Sección de la especificación:	2.6
Descripción:	Formato de la instrucción: Al ejecutar esta instrucción se guarda el dato del registro <i>rs2</i> en la dirección efectiva de la memoria de datos calculada con el valor en el registro <i>rs1</i> más el <i>offset</i> con signo en el <i>imm</i>. Con el <i>funct3</i> se selecciona si el dato leer es un <i>word</i> (32 bits), <i>half</i> (16 bits) o <i>byte</i> (8 bits)
Prioridad:	Crítica
Autor (Dueño de la función):	Sebastian Herrera Esquivel
Requerimientos del banco de pruebas:	Se debe restringir la aleatorización para que la suma del valor en <i>rs1</i> y el <i>offset</i> , que da como resultado la dirección efectiva, esté dentro del rango de la memoria de datos, el cual es [2048, 4096]
Pruebas:	Se coloca previo a la instrucción <i>store</i> una instrucción <i>addi</i> y una <i>slli</i> para colocar un valor dentro del rango en el registro <i>base</i> , también se verifica que no se salga del rango al sumarse con el <i>offset</i>
Criterio de chequeo:	El valor guardado en la dirección efectiva de la memoria de datos debe de coincidir con el valor guardado en el registro fuente, el cual debe ser el mismo calculado por el modelo de referencia.
Cobertura funcional:	Verificar todas las instrucciones de carga (SB, SH, SW), los valores de los registros fuente y destino, y los valores inmediatos con signo. Además, cubrir todos los posibles valores de datos a guardar, y asegurar la cobertura de direcciones de memoria entre 512 y 1023.
Observaciones:	

12. Función #5

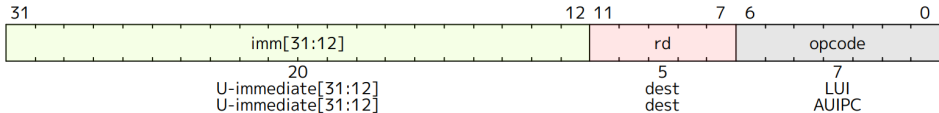
Función a verificar:	Instrucciones tipo BRANCH
Sección de la especificación:	2.5.2
Descripción:	Formato de la instrucción: Cuando el salto se toma, se suma el doble del <i>offset</i> con signo al actual PC, lo que permite un rango de salto de ± 4 KiB.
Prioridad:	Crítica
Autor (Dueño de la función):	Jesús Jiménez
Requerimientos del banco de pruebas:	Se coloca previo a la instrucción <i>branch</i> dos instrucciones <i>addi</i> para colocar un valor dentro del rango en el registro <i>base</i> , también se verifica que no se salga del rango al sumarse con el <i>offset</i>
Pruebas:	El resultado de la comparación entre <i>rs1</i> y <i>rs2</i> guardado en <i>rd</i> se compara con el resultado de la predicción realizada por el modelo de referencia.
Criterio de chequeo:	La condición entre <i>rs1</i> y <i>rs2</i> , el valor inmediato enviado y la predicción de salto coinciden entre el riscv-v y el modelo de referencia.
Cobertura funcional:	Se cubren las siguientes combinaciones: los tipos de instrucciones BEQ a BGEU, los punteros y valores de los registros fuente 1 y 2 (tanto con signo como sin signo), el puntero del registro destino, y los valores inmediatos (con y sin signo). También se incluyen cruces entre tipo de instrucción y punteros de registros fuente, entre tipo de instrucción y registro destino, y entre inmediatos con signo, valores del PC y tipos de instrucciones
Observaciones:	

13. Función #6

Función a verificar:	Instrucciones tipo Jump
Sección de la especificación:	2.5.1
Descripción:	Formato de la instrucción: JAL: El inmediato J codifica un desplazamiento en múltiplos de 2 bytes. Este desplazamiento se extiende por signo y se suma a la dirección de la instrucción de salto para formar la dirección de destino del salto. JALR Se calcula la dirección de destino sumando el inmediato de 12 bits con signo al registro <i>rs1</i>, estableciendo el bit menos significativo en cero. La dirección de la

	siguiente instrucción (pc+4) se guarda en el registro rd, pudiendo usar x0 si no se necesita el resultado.
Prioridad:	Crítica
Autor (Dueño de la función):	Dennis Chavarría
Requerimientos del banco de pruebas:	El estímulo debe generar una secuencia aleatoria de números no repetidos que representa la dirección de siguiente salto. Puede generar JAL continuos en el espacio de memoria, pero siempre que se genere un JALR deberá incluir un ADDI antes de él para guardar en un registro un número que se usa para construir la dirección de siguiente salto. Es necesario limitar los valores inmediatos en un rango válido que permita que el PC calculado siempre esté entre 0 y 2044, esto se debe a que ese es el espacio direccionable de instrucciones.
Pruebas:	Se utiliza una prueba dirigida que genera únicamente JAL y JALR que pueden generar offsets positivos o negativos.
Criterio de chequeo:	El checker deberá verificar que el PC del RTL coincide con lo que calcula el scoreboard.
Cobertura funcional:	Puntos de cobertura de instrucción, de registro rs1, de valor inmediato, de PC de salto, intersección de los anteriores,
Observaciones:	

14.Función #7

Función a verificar:	Instrucciones tipo U
Sección de la especificación:	2.4.1
Descripción:	Formato de la instrucción:  LUI (cargar inmediato superior) se utiliza para construir constantes de 32 bits y utiliza el formato de tipo U. LUI coloca el valor inmediato U de 32 bits en el registro de destino rd, llenando los 12 bits más bajos con ceros. AUIPC (añadir inmediato superior al PC) se usa para construir direcciones relativas al PC y también utiliza el formato de tipo U. AUIPC forma un desplazamiento de 32 bits a partir del inmediato U, llenando los 12 bits más bajos con ceros, añade este desplazamiento a la dirección de la instrucción AUIPC y coloca el resultado en el registro rd.
Prioridad:	Crítica
Autor (Dueño de la función):	Ricardo Mena
Requerimientos del banco de pruebas:	Se deben de generar instrucciones AUIPC y LUI de manera que sus <i>rd</i> e <i>imm</i> sean completamente aleatorios.

Pruebas:	El resultado guardado en rd se compara con el resultado de la predicción realizada por el modelo de referencia y se compara el punto de salto a realizar con el del modelo de referencia.
Criterio de chequeo:	Se toma la dirección de registros rd , y su valor, así como el valor inmediato. Se compara con el resultado de la predicción realizada por el modelo de referencia y se compara el punto de salto a realizar con el del modelo de referencia.
Cobertura funcional:	Puntos de tipo de instrucción instrucciones LUI y AUIPC, Valores inmediatos (inmediato con signo), registros de destino, valores del contador del programa, cruces: Examina la combinación de instrucciones con los registros de destino y cruces específicos entre los valores inmediatos, valores de PC y el tipo de instrucción AUIPC.
Observaciones:	

15.Función #8

Función a verificar:	Instrucciones Combinadas
Sección de la especificación:	2.4
Descripción:	Esta función de verificación implementa la aleatorización total del estímulo, lo que logra generar instrucciones R, I, I-Load, Store, Branch, Jump y U.
Prioridad:	Prioridad Media
Autor (Dueño de la función):	Jesus Jimenez
Requerimientos del banco de pruebas:	Comunicación <i>driver-sequencer</i> con el espacio de memoria. La secuencia debe generar todas las instrucciones con las consideraciones vistas en las funciones #1 a la #7. Es necesario evitar que los jumps tengan offset negativos para optimizar la aleatorización y evitar bucles.
Pruebas:	Una única prueba totalmente aleatorizada a la que se le varía la semilla.
Criterio de chequeo:	Aplican los mismos chequeos que en todas las funciones de verificación anteriores. El scoreboard deberá comparar que todos los datos coinciden, lo que incluye la dirección de lectura/escritura, el valor guardado en los registros, los saltos y su dirección de destino así como la decisión de tomar o no un salto condicional.
Cobertura funcional:	Aplican todos los puntos de cobertura de todas las instrucciones, lo que incluye cobertura R, I, I-Load, Store, Branch, Jump y U.
Observaciones :	La prueba no genera saltos con offset negativo.

16.Bugs encontrados

1. Se encuentra un bug en la inicialización de los valores del banco de registros, donde solo se inicializaba la mitad de los registros, y los demás quedaban en X.

```
`ifdef SIMULATION
    integer i;
    //BUG finded: Half of regs
    initialized
    //initial for(i=0;i!=16;i=i+1)
    REGS[i] = 0;
    initial for(i=0;i!=16;i=i+1)
    REGS[i] = 0;
`endif
```

2. Se realiza una suma equivocada para determinar el corrimiento en las operaciones SRA y SRAI(línea 373 darkrisc.v)
 - a. Se tenía: `$signed(S1REG)>>>U2REGX[4:0];`
 - b. Se reemplaza por `$signed( $signed(S1REG)>>>S2REGX[4:0] );`

17.Parches de funcionalidad

1. Parche a la sección de código no sintetizable del darkrisc.v que realiza el readmemh para cargar la memoria. Esto se realiza para evitar que el RISCv acceda antes de que la memoria se haya cargado. Esto se justifica por medio de:
 - a. La idea es que el RISCv continúe haciendo su carga de memoria independientemente, no que la haga algún bloque del ambiente.
 - b. Se conversó que el driver no debería hacer el readmemh.
 - c. Es una forma poco intrusiva sin crear “forces”, accesos jerárquicos o interfaces virtuales